PRACTICAL 1A

Aim: Write a Java Program to create simple calculator.

Input:

```
import java.util.Scanner;

public class SimpleCalculator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter first number: ");
      double a = sc.nextDouble();

      System.out.print("Enter second number: ");
      double b = sc.nextDouble();

      System.out.print("Enter operator (+, -, *, /): ");
      char op = sc.next().charAt(0);

      double result = 0;

      switch (op) {
         case '+':
            result = a + b;
            break;
         case '-':
            result = a - b;
            break;
         case '*':
            result = a * b;
            break;
         case '/':
            if (b != 0)
               result = a / b;
            else
               System.out.println("Error: Division by zero!");
```

```
            break;
        default:
            System.out.println("Invalid operator!");
            return;
    }

    System.out.println("Result = " + result);
  }
}
```

Output:

Enter first number: 10

Enter second number: 5

Enter operator (+, -, *, /): *

Result = 50.0

PRACTICAL 1B

Aim: Write a Java Program to to make a class called as circle. It should have three methods namely: accept radius, calculate area and display the area.

Input:

```
import java.util.Scanner;

class Circle{
   double radius, area;

   void acceptRadius(){
      Scanner sc=new Scanner(System.in);
      System.out.print("Enter radius: ");
      radius= sc.nextDouble();
   }

   void calculateArea(){
      area=3.14*radius*radius;
   }

   void displayArea(){
      System.out.println("Area of circle= "+ area);
   }
}

public class CircleDemo{
   public static void main(String[] args){
      Circle c=new Circle();
      c.acceptRadius();
      c.calculateArea();
      c.displayArea();
   }
}
```

Output:

```
Enter Radius: 5
Area = 78.5
```

PRACTICAL 2A

Aim: Write a Java program to demonstrate use of Access specifiers.

Input:

```
class AccessSpecifier {
   private String a=("Member 1");
   String b=("Member 2");
   protected String c=("Member 3");
   public String d=("Member 4");

   // This will help to print a
   public void printA(){
      System.out.println(a+ " is Private Member");
   }
}

public class AccessExample {
   public static void main(String[] args){
      Access obj=new Access();

      obj.printA();

      System.out.println(obj.b+ " is Default Member");
      System.out.println(obj.c+ " is Protected Member");
      System.out.println(obj.d+ " is Public Member");
   }
}
```

Output:

```
Member 1 is Private Member
Member 2 is Default Member
Member 3 is Protected Member
Member 4 is Public Member
```

PRACTICAL 2B

Aim: Write a Java program to demonstrate use of access specifiers.

1.Date and Time:

Input:

```
import java.util.*;
class DateTime
{
   public static void main(String args[])
   {
      Date d= new Date();
      System.out.println(d);
   }
}
```

Output:

Fri Jul 11 21:54:19 GMT 2025

2.Calender:

Input:

```
import java.util.*;
class DateTime
{
   public static void main(String args[])
   {
      String monthName[]= {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug",
"Sep", "Oct", "Nov", "Dec"};
      Calendar c= Calendar.getInstance();
      System.out.println("Date:"
+c.get(Calendar.DATE)+"/"+monthName[c.get(Calendar.MONTH)]+"/"+c.get(Calen
dar.YEAR));
      System.out.println("Time:"+c.get(Calendar.HOUR)+
":"+c.get(Calendar.MINUTE)+":"+c.get(Calendar.SECOND));
   }
}
```

Output:

```
Date:11/Jul/2025
Time:9:56:18
```

PRACTICAL 3A

Aim: Write a Java Program to demonstrate ArithmeticException and NumberFormatException using user input in Java.

Input:

```
import java.util.Scanner;

public class ExceptionDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         // NumberFormatException example
         System.out.print("Enter a number: ");
         String str = sc.next();         // user enters input
         int num = Integer.parseInt(str); // may throw NumberFormatException

         // ArithmeticException example
         System.out.print("Enter divisor: ");
         int divisor = sc.nextInt();
         int result = num / divisor;      // may throw ArithmeticException
         System.out.println("Result = " + result);

      } catch (NumberFormatException e) {
         System.out.println("Error: Invalid number format!");
      } catch (ArithmeticException e) {
         System.out.println("Error: Division by zero not allowed!");
      }
   }
}
```

First Output:
Enter a number: 1
Enter divisor: 2
Result = 0

Second Output:
Enter a number: a
ERROR!
Error: Invalid number format!

Third Output:
Enter a number: 2
Enter divisor: 0
ERROR: Division by zero not allowed!

PRACTICAL 3B

Aim: Write a Java Program to perform division of two user-entered numbers and handle NumberFormatException and ArithmeticException using try-catch blocks.

Input:

```
import java.util.Scanner;

public class DivisionExample {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         // input numbers as strings
         System.out.print("Enter first number: ");
         String str1 = sc.next();
         System.out.print("Enter second number: ");
         String str2 = sc.next();

         // may throw NumberFormatException
         int num1 = Integer.parseInt(str1);
         int num2 = Integer.parseInt(str2);

         // may throw ArithmeticException
         int result = num1 / num2;

         System.out.println("Result = " + result);

      } catch (NumberFormatException e) {
         System.out.println("Error: Please enter valid numbers!");
      } catch (ArithmeticException e) {
         System.out.println("Error: Division by zero is not allowed!");
      }
   }
}
```

First Output:

Enter the first Number: 32
Enter the second Number: 35
The Quotient = 0

Second Output:

Enter the first Number: 35
Enter the second Number: 0
ERROR!
Error: Division by zero is not allowed.

Third Output:

Enter First Number: 35
Enter Second Number: S
Invalid Number. Please enter only integers.

PRACTICAL 3C

Aim: Write a Java program to accept and display the month number. Throw a number format exception if improper month number is entered.

Input:

```
import java.io.*;
class month
{
   public static void main(String args[])throws IOException
   {
      int m;
      BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
      String str;
      System.out.println("Enter month number: ");
      str= br.readLine();
      m= Integer.parseInt(str);
      try
      {
         if(m>12||m<1)
         throw new NumberFormatException();
         System.out.println("Month number entered is " + m);
      }
      catch(NumberFormatException ne)
      {
         System.out.println("Invalid month number! ");
      }
   }
}
```

First Output:

```
Enter month number:
5
Month number entered is 5
```

Second Output:

```
Enter month number:
35
Invalid month number!
```

PRACTICAL 4A

Aim: Write two Java programs to print numbers from 1 to 10 using:

1. **By implementing the Runnable interface.**
2. **By extending the Thread class.**

Input: By implementing the Runnable interface.

```
class Numbers implements Runnable {
   public void run() {
      for (int i = 1; i <= 10; i++) {
         System.out.println(i);
      }
   }
}

public class Main {
   public static void main(String[] args) {
      Numbers n = new Numbers();
      Thread t1 = new Thread(n);
      t1.start();
   }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Input: By extending the Thread class.

```
class Numbers extends Thread {
   public void run() {
      for (int i = 1; i <= 10; i++) {
         System.out.println(i);
      }
   }
}

public class Main {
   public static void main(String[] args) {
      Numbers n = new Numbers();
      Thread t1 = new Thread(n);
      t1.start();
   }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

PRACTICAL 4B

Aim: Write Java programs to print even and odd numbers from 1 to 50 using:

1. **Runnable interface.**
2. **Extending the Thread class.**

Input: By implementing the Runnable interface.

```
class Numbers implements Runnable{
   public void run() {
      for (int i=1; i<=50; i++){
         if (i%2==0) {
            System.out.println (i + ":is an even number.");
         } else
         {
            System.out.println (i + ":is an odd number.");
         }
      }
   }
}

public class Main {
   public static void main(String[] args) {
      Numbers n= new Numbers();
      Thread t1= new Thread(n);
      t1.start();
   }
}
```

Output:

1:is an odd number.
2:is an even number.
3:is an odd number.
4:is an even number.
5:is an odd number.
6:is an even number.
7:is an odd number.
8:is an even number.
9:is an odd number.
10:is an even number.
11:is an odd number.
12:is an even number.
13:is an odd number.
14:is an even number.
15:is an odd number.
16:is an even number.
17:is an odd number.
18:is an even number.
19:is an odd number.
20:is an even number.
21:is an odd number.
22:is an even number.
23:is an odd number.
24:is an even number.
25:is an odd number.
26:is an even number.
27:is an odd number.
28:is an even number.
29:is an odd number.
30:is an even number.
31:is an odd number.
32:is an even number.
33:is an odd number.
34:is an even number.
35:is an odd number.
36:is an even number.
37:is an odd number.
38:is an even number.
39:is an odd number.
40:is an even number.
41:is an odd number.
42:is an even number.
43:is an odd number.
44:is an even number.
45:is an odd number.
46:is an even number.
47:is an odd number.
48:is an even number.
49:is an odd number.
50:is an even number.

Input: By extending the Thread class.

```
class Numbers extends Thread{
   public void run() {
      for (int i=1; i<=50; i++){
         if (i%2==0) {
            System.out.println (i + ":is an even number.");
         } else
         {
            System.out.println (i + ":is an odd number.");
         }
      }
   }
}

public class Main {
   public static void main(String[] args) {
      Numbers n= new Numbers();
      Thread t1= new Thread(n);
      t1.start();
   }
}
```

Output:

```
1:is an odd number.
2:is an even number.
3:is an odd number.
4:is an even number.
5:is an odd number.
6:is an even number.
7:is an odd number.
8:is an even number.
9:is an odd number.
10:is an even number.
11:is an odd number.
12:is an even number.
13:is an odd number.
14:is an even number.
15:is an odd number.
16:is an even number.
17:is an odd number.
18:is an even number.
19:is an odd number.
20:is an even number.
21:is an odd number.
22:is an even number.
23:is an odd number.
24:is an even number.
25:is an odd number.
26:is an even number.
27:is an odd number.
28:is an even number.
29:is an odd number.
30:is an even number.
31:is an odd number.
32:is an even number.
33:is an odd number.
34:is an even number.
35:is an odd number.
36:is an even number.
37:is an odd number.
38:is an even number.
39:is an odd number.
40:is an even number.
41:is an odd number.
42:is an even number.
43:is an odd number.
44:is an even number.
45:is an odd number.
46:is an even number.
47:is an odd number.
48:is an even number.
49:is an odd number.
50:is an even number.
```

PRACTICAL 4C

Aim: Write a Java Program to demonstrate isAlive() and join() method.

Input:

```
class Alphabets extends Thread {
   public void run() {
      for (int i = 1; i <= 2; i++) {
         System.out.println((char)(i + 64));
         try {
            Thread.sleep(100);
         } catch (Exception e) {

         }
      }
   }
}

class Numbers extends Thread {
   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(i);
         try {
            Thread.sleep(100);
         } catch (Exception e) {

         }
      }
   }
}

public class Main {
   public static void main(String[] args) {
      Alphabets n = new Alphabets();
      Thread t1 = new Thread(n);

      Numbers n1 = new Numbers();
```

```
        Thread t2 = new Thread(n1);

        t1.start();
        t2.start();

        System.out.println("Thread t1 is alive: " + t1.isAlive());
        System.out.println("Thread t2 is alive: " + t2.isAlive());

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException ie) {

        }

        System.out.println("Thread t1 is alive: " + t1.isAlive());
        System.out.println("Thread t2 is alive: " + t2.isAlive());
    }
}
```

Output:

```
A
1
Thread t1 is alive: true
Thread t2 is alive: true
B
2
3
4
5
Thread t1 is alive: false
Thread t2 is alive: false
```

PRACTICAL 5A

Aim: Write a Java program to demonstrate the use of methods in linked list instance.

Input:

```
import java.util.*;

class LinkedListDemo {
   public static void main(String ar[]) {
      LinkedList<String> list = new LinkedList<>();

      list.add("One");
      list.add("Two");
      list.add("Three");
      list.add("Four");
      list.add("Five");
      list.addLast("Six");
      list.addFirst("Seven");
      list.add(1, "Eight");

      System.out.println("Original Contents of list: " + list);

      list.remove("Five");
      list.remove(2);
      System.out.println("\nContents of list after deletion: " + list);

      list.removeFirst();
      list.removeLast();
      System.out.println("\nList after deleting first and last: " + list);

      String val = list.get(2);
      list.set(2, val+ "-Changed");
      System.out.println("\nList after change: " + list);
   }
```

Output:

Original Contents of list: [Seven, Eight, One, Two, Three, Four, Five, Six]

Contents of list after deletion: [Seven, Eight, Two, Three, Four, Six]

List after deleting first and last: [Eight, Two, Three, Four]

List after change: [Eight, Two, Three-Changed, Four]

PRACTICAL 5B

Aim: Write a Java program to demonstrate the simple use of class ArrayList.

Input:

```
import java.util.*;
class ArrayListDemo {
   public static void main(String ar[]) {
      ArrayList<String> list = new ArrayList<>();
      System.out.println("\nInitilist size of list: "+list.size());

      list.add("First");
      list.add("Second");
      list.add("Three");
      list.add("Four");
      list.add("Five");
      list.add("Six");
      list.add(1, "Seven");

      System.out.println("\nSize of list after additions: " + list.size());

      System.out.println("\nContents of list: "+ list);

      list.remove("Five");
      list.remove(2);
      System.out.println("\nSize of list after deletions: " + list.size());

      System.out.println("\nContents of list: "+list);
   }
```

Output:

Initilist size of list: 0

Size of list after additions: 7

Contents of list: [First, Seven, Second, Three, Four, Five, Six]

Size of list after deletions: 5

Contents of list: [First, Seven, Three, Four, Six]